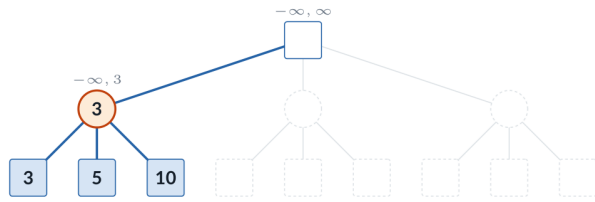


Artificial Intelligence Exam Notes

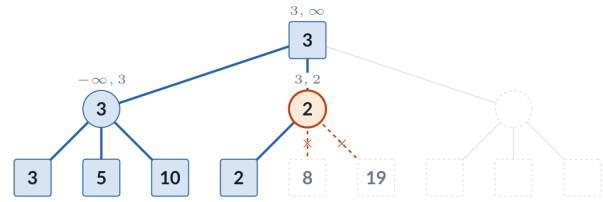
CT 710 (BEI, IV-I) · CT 653 (BCT, III-II) · CT 78506 (BEX, IV-II) · TU, IOE · tiers over 41 papers, 2069–2082 BS

Step by step: alpha-beta, one MIN node at a time

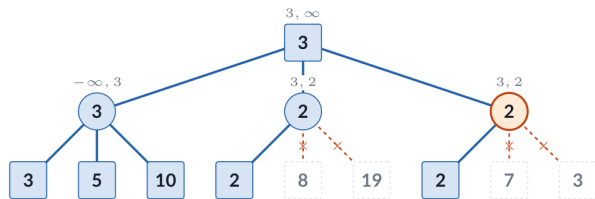
1. $m_1 = \min(3, 5, 10) = 3$



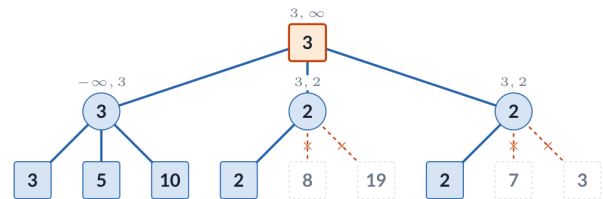
2. $m_2 = \min(2) = 2$ **prune 8, 19**



3. $m_3 = \min(2) = 2$ **prune 7, 3**



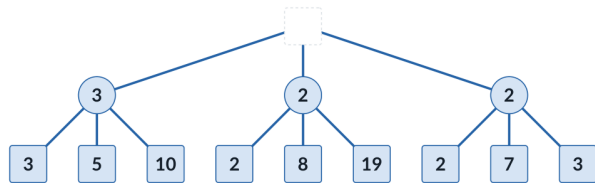
4. $\text{root} = \max(3, 2, 2) = 3$



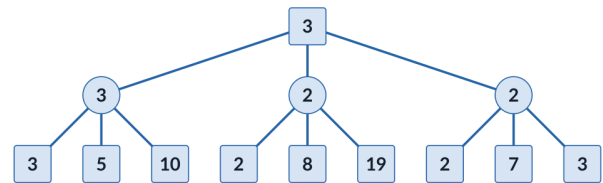
● value backed up ○ entered, α, β set ● just finished ○ pruned

Step by step: minimax backing the values up

1. MIN over each group of three leaves



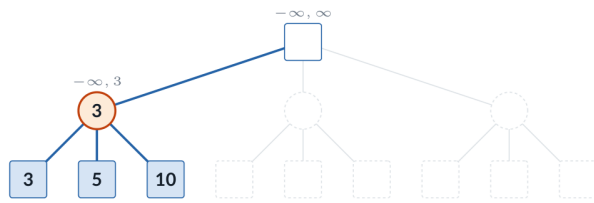
2. MAX at the root



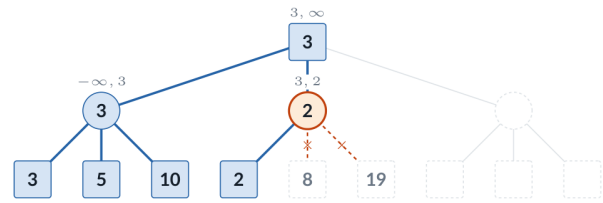
● value backed up ○ entered, α, β set ● just finished ○ pruned

Step by step: alpha-beta, one MIN node at a time

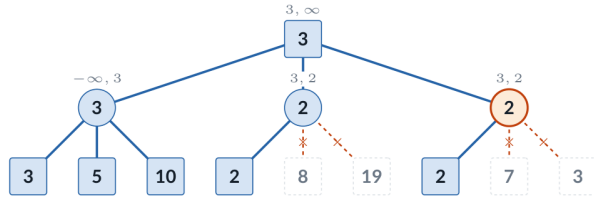
1. $m_1 = \min(3, 5, 10) = 3$



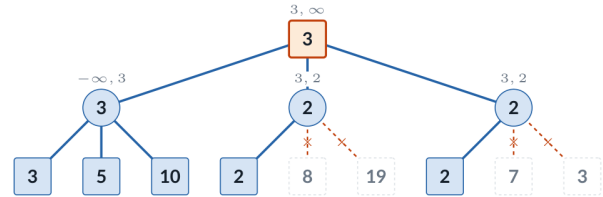
2. $m_2 = \min(2) = 2$ **prune 8, 19**



3. $m_3 = \min(2) = 2$ **prune 7, 3**



4. $\text{root} = \max(3, 2, 2) = 3$



● value backed up
 ○ entered, α, β set
 ○ just finished
 ○ pruned

3. Search Techniques – Solved Problems

Every problem opens with the exam question and the figure **exactly as the paper prints them**. The OPEN / CLOSED table format is the one *Insights* §3.4.2 uses. · Every path, cost and minimax value in this file is recomputed independently by `verify.py`.

The four things everything here is built on:

- 1. $f(n) = g(n) + h(n)$. **Greedy** orders on h alone, **uniform cost** on g alone, **A*** on the sum. Every trace below is the same loop with a different f .
- 2. **The loop**: take the best node off OPEN → put it on CLOSED → generate its successors onto OPEN → **re-order OPEN** → repeat. Stop when the node taken off OPEN is the goal, not when the goal is first *generated*.
- 3. **If a successor is already known with a larger g , replace it** and put it back on OPEN. Skipping this returns the wrong answer on both A* graphs in these papers, because both heuristics are admissible but **not consistent**.
- 4. **Alpha-beta**: α only rises, β only falls, a child inherits both, and you test the cut rule after *every* child. MIN cuts when its value $\leq \alpha$; MAX cuts when its value $\geq \beta$.

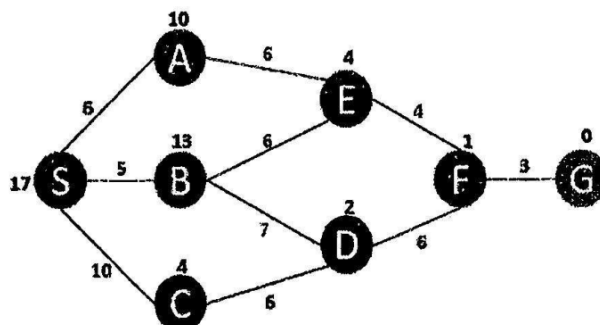
3.N.1 A* on the 2081 Chaitra graph

Explain the A* algorithm on the given figure 8

PIN 1/41 81 Ch 8 · the number **above** each node is $h(n)$, the number on each link is the step cost

QUESTION AS PRINTED 81 Ch · 8

Explain the A* algorithm on the following figure, where the starting node is S and the destination is G. (Information in the node represents the cost required to reach the goal and information in the connecting link represents the cost from one node to the other.)



Reading the figure

- h : $S=17, A=10, B=13, C=4, D=2, E=4, F=1, G=0$.
- Edges: $S-A$ 6, $S-B$ 5, $S-C$ 10, $A-E$ 6, $B-E$ 6, $B-D$ 7, $C-D$ 6, $E-F$ 4, $D-F$ 6, $F-G$ 3.
- There are only four routes from S to G , which is worth writing down first as a check on the answer:
 - $S-A-E-F-G = 6+6+4+3 = 19$
 - **$S-B-E-F-G = 5+6+4+3 = 18$**
 - $S-B-D-F-G = 5+7+6+3 = 21$
 - $S-C-D-F-G = 10+6+6+3 = 25$
- **So the answer must come out 18.** If your trace ends at 19 you have skipped the reopening step.

Step 1. Start with S on OPEN

OPEN				CLOSED	
Node	g	h	f	Node	Parent
S	0	17	17		

Step 2. Expand S

OPEN, re-ordered by f				CLOSED	
Node	g	h	f	Node	Parent
C	10	4	14	S	—
A	6	10	16		
B	5	13	18		

- **Note what has already gone wrong for greedy:** C is chosen first because $h(C) = 4$ is tiny, yet C lies on the worst route. A^* will recover; greedy would not.

Steps 3–6. Expand C, A, E, F

Expand	Generates	OPEN after re-ordering (node/ f)
C (f 14)	D: $g=16, f=18$	A/16, B/18, D/18
A (f 16)	E: $g=12, f=16$	E/16, B/18, D/18
E (f 16)	F: $g=16, f=17$	F/17, B/18, D/18
F (f 17)	G: $g=19, f=19$ D via F: $g=22$, worse, discard	B/18, D/18, G/19

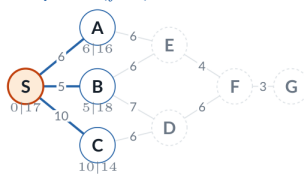
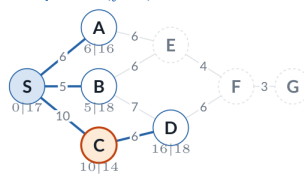
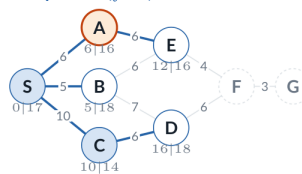
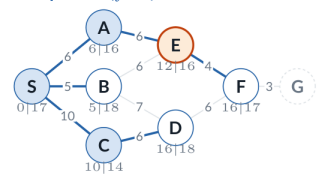
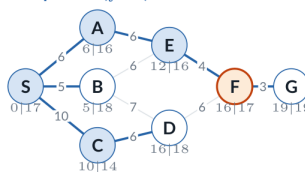
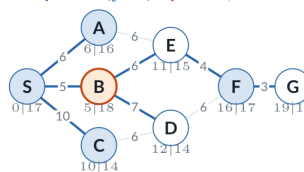
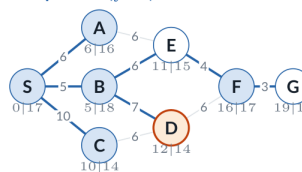
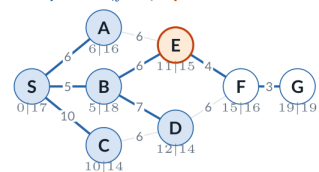
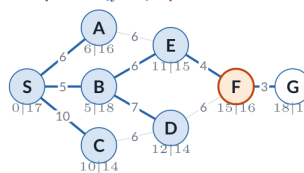
- G is now **generated** at cost 19, but it is **not expanded**, because B and D both have $f = 18 < 19$. This is exactly why the algorithm stops on *removal* from OPEN, not on generation.

Steps 7–10. Expanding B revises everything

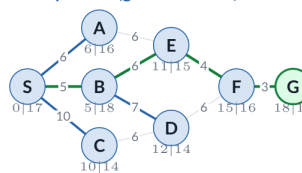
Expand	Generates	OPEN after re-ordering
B (f 18)	E: $g=11 < 12$, replace , $f=15$ D: $g=12 < 16$, replace , $f=14$	D/14, E/15, G/19
D (f 14)	F via D: $g=18 > 16$, discard	E/15, G/19
E (f 15)	F: $g=15 < 16$, replace , $f=16$	F/16, G/19
F (f 16)	G: $g=18 < 19$, replace , $f=18$	G/18
G (f 18)	goal removed from OPEN, stop	

- **Three replacements**, all through B . B looked unpromising ($f = 18$, last in the queue) but is on the optimal route.

Step by step: the graph after each expansion

1. expand S (f 17)2. expand C (f 14)3. expand A (f 16)4. expand E (f 16)5. expand F (f 17)6. expand B (f 18) replace D, E7. expand D (f 14)8. expand E (f 15) replace F9. expand F (f 16) replace G

10. expand G (goal off OPEN)



● closed
 ○ on OPEN
 ○ expanding now
 ○ goal

Answer: path $S \rightarrow B \rightarrow E \rightarrow F \rightarrow G$, total cost **18**. Nodes expanded in the order S, C, A, E, F, B, D, E, F, G.

Why this graph is worth understanding rather than memorising: h here is *admissible* (it never exceeds the true remaining cost) but *not consistent* – $h(B) = 13$ while $\text{cost}(B, E) + h(E) = 6 + 4 = 10$. With an inconsistent heuristic a node can be reached more cheaply after it has been closed, which is why E and F each get expanded twice. An implementation with a strict closed list and no reopening returns $S-A-E-F-G$ at cost 19, and that is the commonest wrong answer to this question.

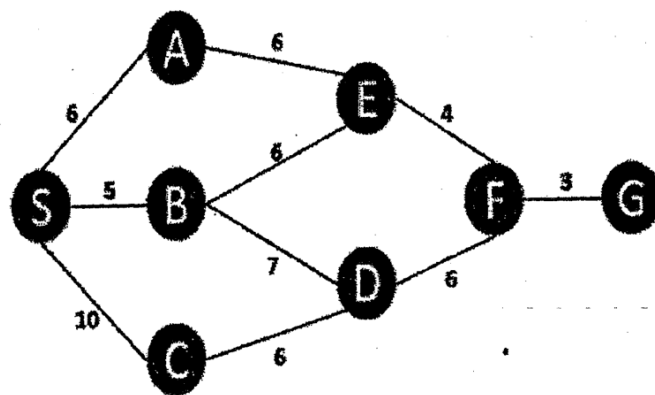
3.N.2 A* Vs greedy on the 2080 Chaitra graph

How does A* overcome the problem with greedy best-first search? **3+4+3**

PIN 1/41 80 Ch **3+4+3** · the same graph as §3.N.1 with different heuristics, given in a table rather than on the nodes. Check which you have been given before starting

QUESTION AS PRINTED 80 Ch · **3+4+3**

How does A* search overcome the problem associated with Greedy Best First Search? Using the following figure and table, where the starting node is S and the destination is G, compare the result of the A* algorithm with greedy search. (The table gives the cost required to reach the goal; the connecting link gives the cost from one node to the other.)



From	S	A	B	C	D	E	F	G
Cost	15	10	12	5	4	2	1	0

Greedy best first: order on h alone

Expand	OPEN after (h)	CLOSED
S	C/5, A/10, B/12	S
C	D/4, A/10, B/12	S, C
D	F/1, A/10, B/12	S, C, D
F	G/0, E/2, A/10, B/12	S, C, D, F
G	goal	S, C, D, F, G

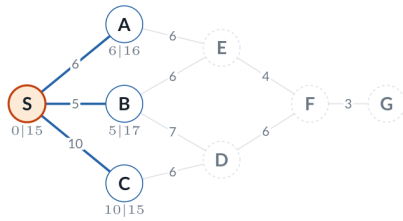
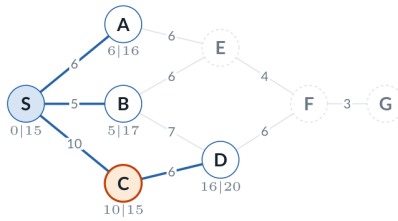
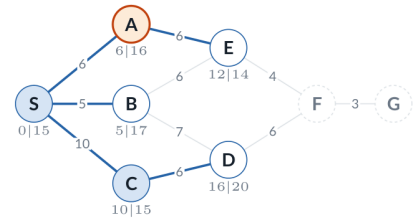
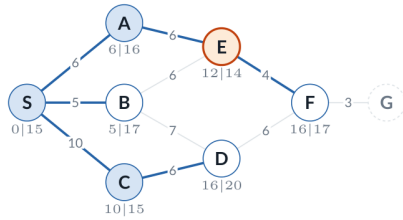
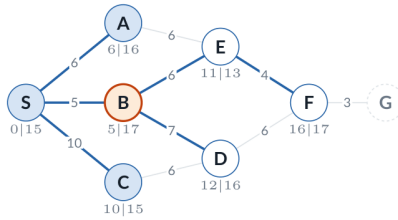
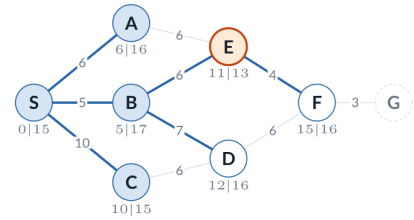
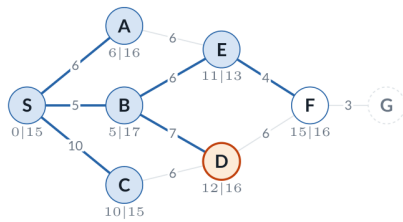
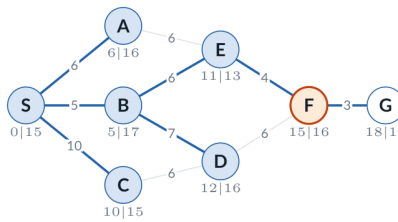
- Path $S \rightarrow C \rightarrow D \rightarrow F \rightarrow G$, cost $10 + 6 + 6 + 3 = 25$.
- **That is the worst of the four routes.** Greedy went straight to C because $h(C) = 5$ is the smallest number in sight, never noticing that *getting* to C costs 10, twice what B costs.
- Only 5 nodes expanded, so it is fast. It is fast and wrong.

Step by step: A* on the same graph

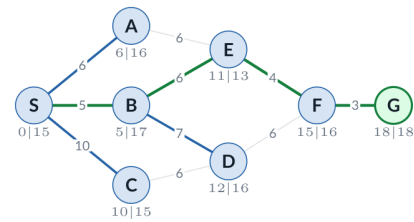
A*: order on $f = g + h$

Expand	g, h, f	OPEN after re-ordering
S	0, 15, 15	C/15, A/16, B/17
C	10, 5, 15	A/16, B/17, D/20
A	6, 10, 16	E/14, B/17, D/20
E	12, 2, 14	B/17, F/17, D/20
B	5, 12, 17	E/13, D/16, F/17
E	11, 2, 13	D/16, F/16
D	12, 4, 16	F/16
F	15, 1, 16	G/18
G	18, 0, 18	goal, stop

- Path $S \rightarrow B \rightarrow E \rightarrow F \rightarrow G$, cost **18**, the optimum.

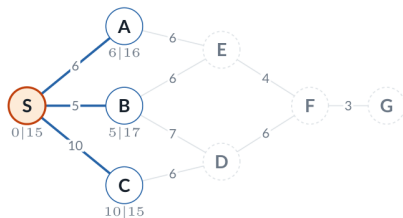
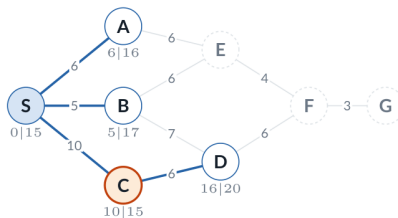
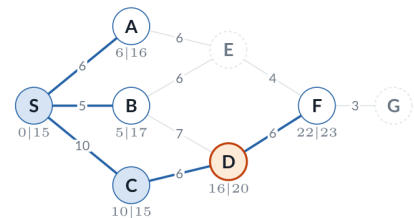
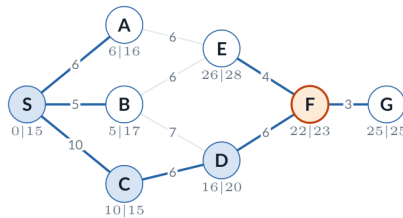
1. expand S (f 15)2. expand C (f 15)3. expand A (f 16)4. expand E (f 14)5. expand B (f 17) replace D,E6. expand E (f 13) replace F7. expand D (f 16)8. expand F (f 16)

9. expand G (goal off OPEN)

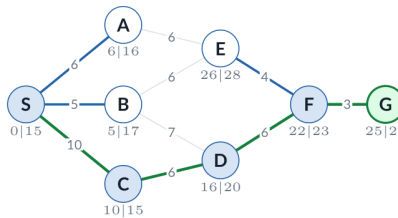


● closed ● on OPEN ● expanding now ● goal

Step by step: greedy best first

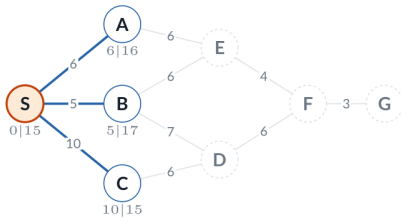
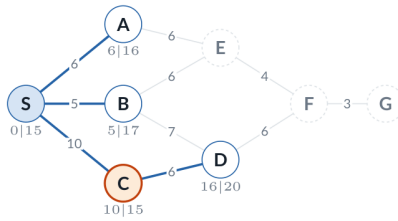
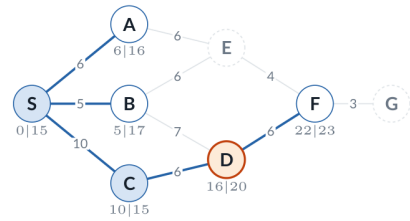
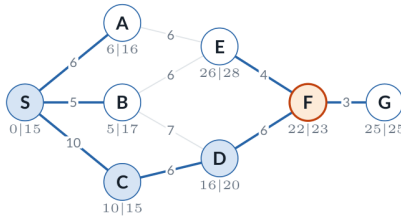
1. expand S (h 15)2. expand C (h 5)3. expand D (h 4)4. expand F (h 1)

5. expand G (goal)

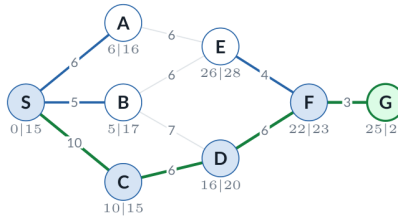


● closed ● on OPEN ● expanding now ● goal

Step by step: greedy best first

1. expand S (h 15)2. expand C (h 5)3. expand D (h 4)4. expand F (h 1)

5. expand G (goal)



● closed ● on OPEN ● expanding now ● goal

The comparison, which is what the 3 marks are for

	Greedy best first	A*
$f(n)$	$h(n)$	$g(n) + h(n)$
Path found	S-C-D-F-G	S-B-E-F-G
Cost	25, the worst route	18, the optimum
Nodes expanded	5	9
Optimal	× no	✓ yes, h is admissible
Complete	× no	✓ yes

- **The trade in one line:** A* expanded almost twice as many nodes and returned a path 28 % cheaper. Greedy buys speed with correctness.

Where greedy went wrong, precisely

- At the very first step both algorithms rank C best, but for different reasons. Greedy sees $h(C) = 5$ and **commits**; A* sees $f(C) = 10 + 5 = 15$, ties with nothing better, expands it, and then **keeps B on the table**.
- Once greedy has moved to C it never reconsiders. It has **no memory of cost paid**, so nothing can ever make C look like a mistake.
- **The general statement to close with:** a heuristic estimates the *remaining* journey. Judging a node on that alone is judging a route by how it ends, ignoring how much it cost to get there. Adding $g(n)$ restores the whole journey to the comparison, and that is the entire difference between the two algorithms.

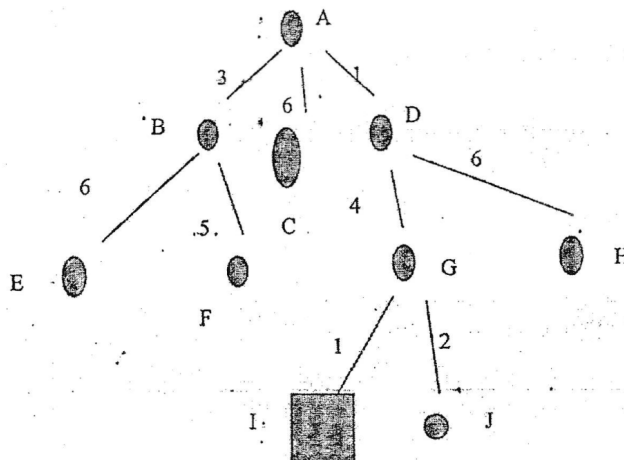
3.N.3 Best first search on the 2076 Bhadra tree

Show the open list, closed list and path at each step 5

76 Bh 5 · 1 paper, 5 marks, so no tier chip · the question names the **three things it wants tabulated**, so tabulate exactly those three

QUESTION AS PRINTED 76 Bh · 5

Consider a search tree for a path-finding problem as shown. Assuming node A to be the root node and node I to be the goal node, explain how Best First Search reaches the goal by showing changes in the open list, closed list, and path at each step of the search.



Reading the figure

- Tree: $A \rightarrow B, C, D$; $B \rightarrow E, F$; $D \rightarrow G, H$; $G \rightarrow I, J$. I (the square) is the goal.
- Values on the edges, taken as the estimate for the child node: $B=3, C=6, D=1, E=6, F=5, G=4, H=6, I=1, J=2$.
- Best first** expands the OPEN node with the smallest value, so the OPEN list is re-ordered after every expansion.

The trace

Step	Expand	OPEN (node/value)	after	CLOSED
1	A	D/1, B/3, C/6	A	
2	D	B/3, G/4, C/6, H/6	A, D	
3	B	G/4, F/5, C/6, E/6, H/6	A, D, B	
4	G	I/1, J/2, F/5, C/6, E/6, H/6	A, D, B, G	
5	I	goal reached	A, D, B, G, I	

Path at each step

Step Path to the node just expanded

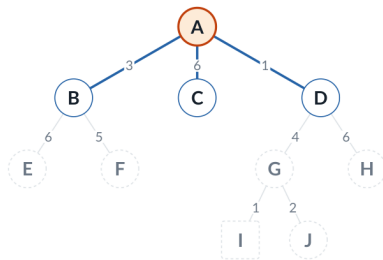
- 1 A
- 2 $A \rightarrow D$
- 3 $A \rightarrow B$
- 4 $A \rightarrow D \rightarrow G$
- 5 $A \rightarrow D \rightarrow G \rightarrow I$

Answer: path $A \rightarrow D \rightarrow G \rightarrow I$, cost $1 + 4 + 1 = 6$.

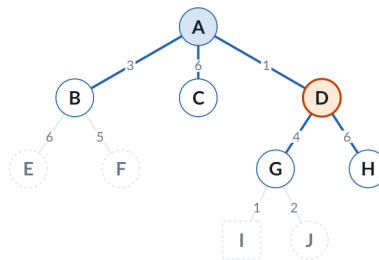
- Step 3 is the one to explain.** After expanding D , the best node on OPEN is B at 3, not G at 4 – so best first **jumps back across the tree** to a different branch before returning to G . That switching is precisely what distinguishes best first from depth first, and it is where the marks are.
- C, E, F, H and J are **never expanded**. Five of the ten nodes are never touched, which is the point of using a heuristic at all.
- If the numbers are read as *edge costs* and accumulated instead, the same path comes out at the same cost of 6. Either reading is defensible here; say which you are using.

Step by step: the tree after each expansion

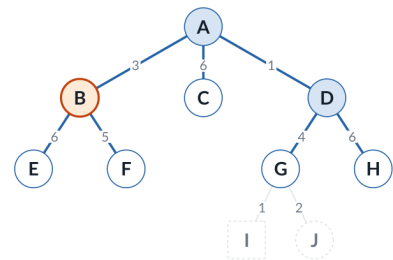
1. expand A (root)



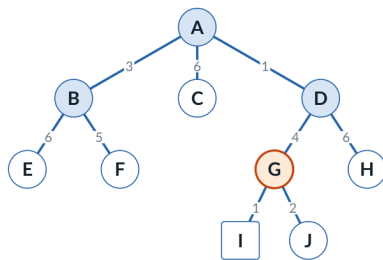
2. expand D (value 1)



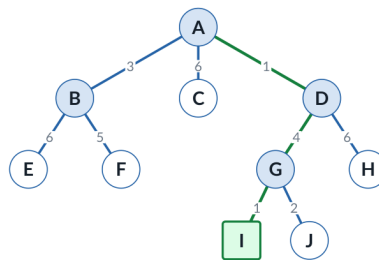
3. expand B (value 3)



4. expand G (value 4)

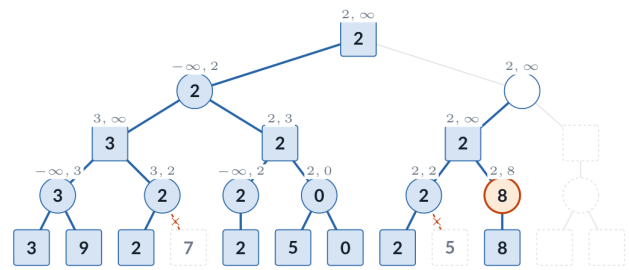
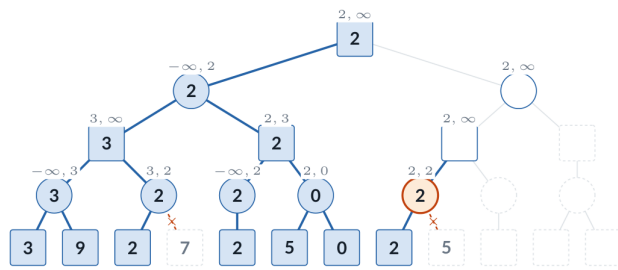


5. expand I (goal)

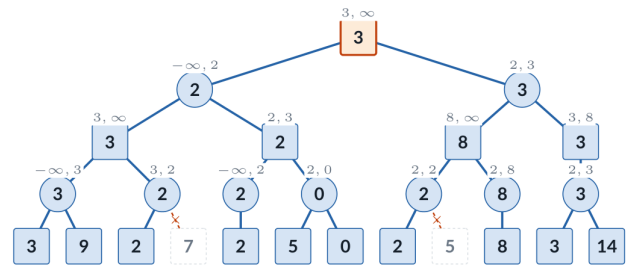
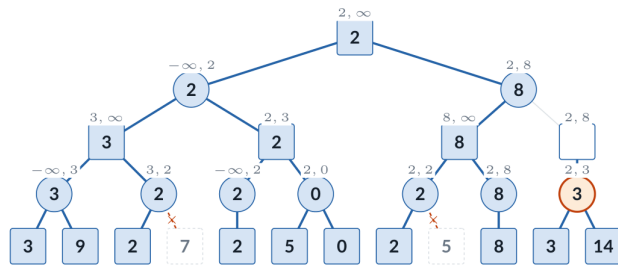


● closed
 ● on OPEN
 ● expanding now
 ■ goal

6. $c_6 = \min(8) = 8$



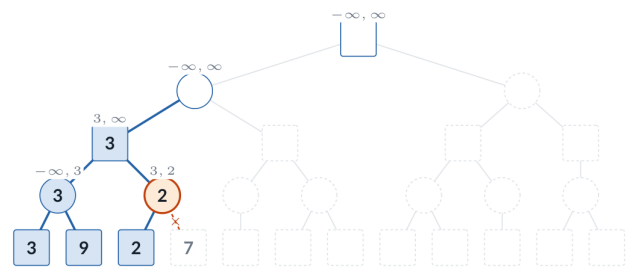
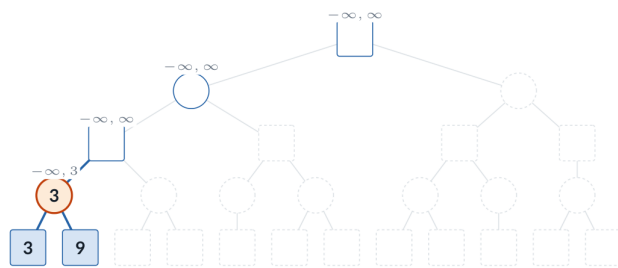
8. $\text{root} = \max(2, 3) = 3$



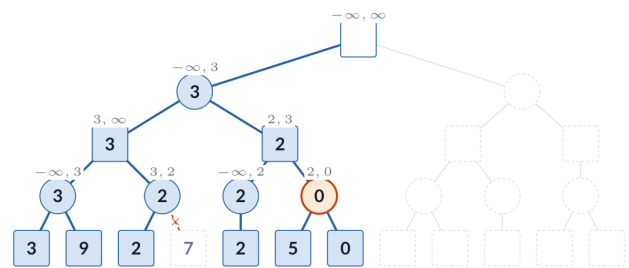
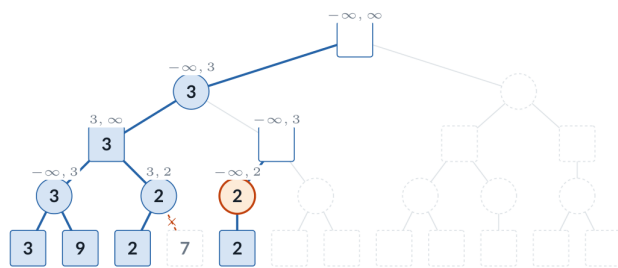
 value backed up
  entered, α , β set
  just finished
  pruned

Step by step: alpha-beta, left half

2. $c_2 = \min(2) = 2$ **prune 7**



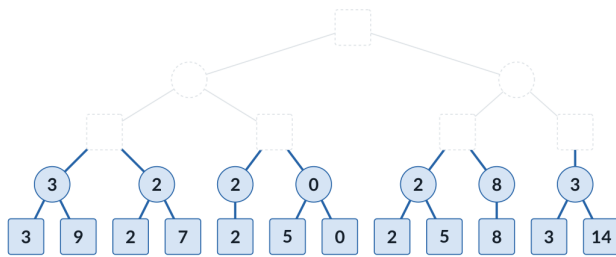
4. $c_4 = \min(5, 0) = 0$



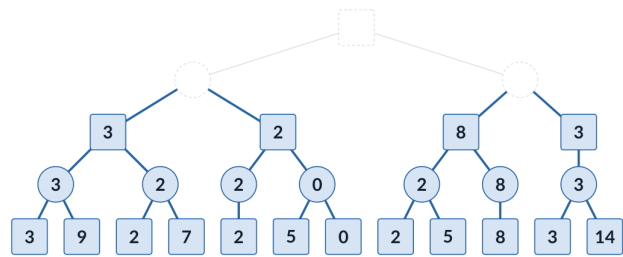
 value backed up
  entered, α , β set
  just finished
  pruned

Step by step: minimax backing the values up

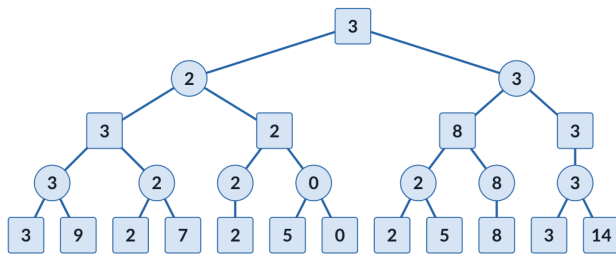
1. MIN over the leaves



2. MAX row

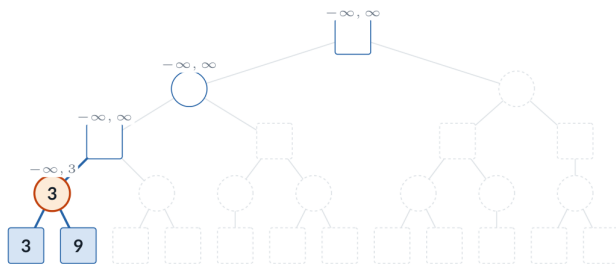
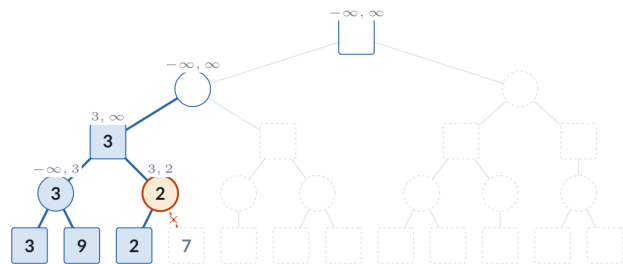
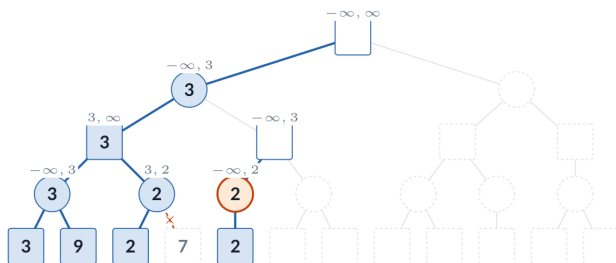
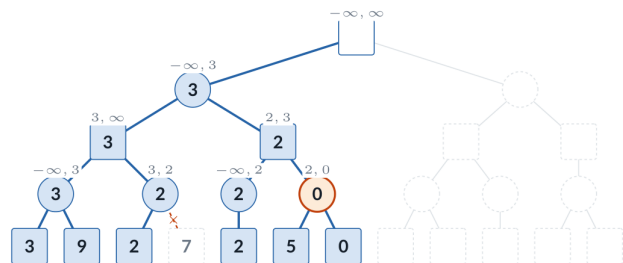


3. MIN row, then the root



● value backed up
 ○ entered, α, β set
 ○ just finished
 ○ pruned

Step by step: alpha-beta, left half

1. $c_1 = \min(3, 9) = 3$ 2. $c_2 = \min(2) = 2$ **prune 7**3. $c_3 = \min(2) = 2$ 4. $c_4 = \min(5, 0) = 0$ 

● value backed up
 ○ entered, α, β set
 ○ just finished
 ○ pruned

Answer: minimax value **3**; MAX plays the right branch. **2 of the 12 leaves are pruned:** the **7** under c_2 , and the **5** under c_5 . Ten leaves are examined.

Note the equality in the second cut. At c_5 the value is 2 and α is also 2. A MIN node cuts on $\text{value} \leq \alpha$, not $<$, because MAX already has a way to score 2 elsewhere and will never *prefer* this branch. Using a strict $<$ here is the commonest slip and it costs you that pruned leaf.

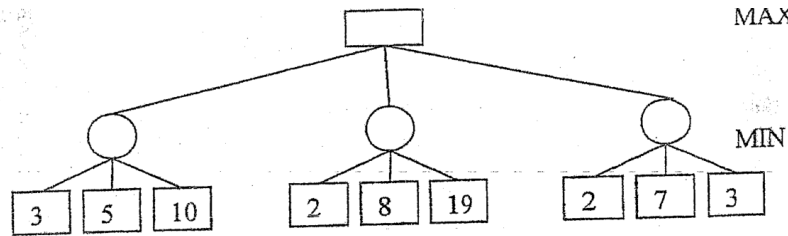
3.N.5 Alpha-beta on the 2076 Baishakh tree

Discuss alpha-beta pruning and find the minimax value 4+4

PIN 1/41 76 Ba 4+4 · a small tree that prunes a much larger fraction than §3.N.4, and the reason why is the lesson

QUESTION AS PRINTED 76 Ba · 4+4

Discuss the alpha-beta pruning algorithm. Find the min-max value using this concept in the following tree.



The trace

- MAX root over three MIN nodes, each with three leaves: $\{3, 5, 10\}$, $\{2, 8, 19\}$, $\{2, 7, 3\}$.

Node	(α, β)	What happens
m_1	$(-\infty, \infty)$	$3 \rightarrow 5 \rightarrow 10$, value = 3. Nothing can be pruned here , because α is still $-\infty$
root	$\alpha \leftarrow 3$	
m_2	$(3, \infty)$	leaf 2 gives 2; $2 \leq 3 \rightarrow$ PRUNE 8 and 19
m_3	$(3, \infty)$	leaf 2 gives 2; $2 \leq 3 \rightarrow$ PRUNE 7 and 3
root		$\max(3, 2, 2) = 3$

- Minimax value 3, MAX plays the **leftmost** branch.
- 4 of the 9 leaves pruned (8, 19, 7, 3); five examined: 3, 5, 10, 2, 2.

Why this tree prunes so much better

- §3.N.4 pruned 2 leaves in 12, about 17%. This one prunes 4 in 9, about 44%. The algorithm is identical; the **move ordering** is not.
- Here the **best branch happens to be examined first**. That sets $\alpha = 3$ immediately, and every later MIN node that finds any child below 3 can stop at once. Both of them do, on their very first leaf.
- **The general rule:** alpha-beta is only as good as the order in which moves are examined.
 - **Best case** (best move first everywhere): $O(b^{m/2})$, so the same time searches **twice as deep**.
 - **Worst case** (best move last everywhere): $O(b^m)$, i.e. **no saving at all** – alpha-beta never prunes more than minimax examines, and can prune nothing.
- This is why real game programs spend effort on **move ordering heuristics** before they spend it on searching deeper.

Two things to say when asked to “discuss” it

- **It returns exactly the minimax value.** Pruning removes work, never accuracy. Say this first.
- A pruned branch is one whose value **cannot change the root**, because a player higher up already has a better guaranteed alternative.

Abbreviations used in these notes

Only these are ever shortened. Anything not on this list is written out in full.

Short	Means	Short	Means
AI	artificial intelligence	ANI	artificial narrow intelligence
AGI	artificial general intelligence	ASI	artificial super intelligence
KR	knowledge representation	KRR	knowledge representation and reasoning
PEAS	performance measure, environment, actuators, sensors	GPS	General Problem Solver
NLP	natural language processing	NLU	natural language understanding
NLG	natural language generation	CAPTCHA	completely automated public Turing test to tell computers and humans apart
ML	machine learning	ANN, NN	artificial neural network
MLP	multilayer perceptron	BP	back-propagation
SOM	self-organizing map	GA	genetic algorithm
FIS	fuzzy inference system	ID3	Iterative Dichotomiser 3
CSP	constraint satisfaction problem	DFS	depth first search
BFS	breadth first search	DLS	depth limited search
IDS	iterative deepening search	UCS	uniform cost search
CNF	conjunctive normal form	WFF	well-formed formula
FOL	first-order logic	FOPL	first-order predicate logic
RRS	resolution refutation system	BBN	Bayesian belief network

Short	Means	Short	Means
ES	expert system	KB	knowledge base
CD	conceptual dependency	LISP	list processing (the language)
MOP	memory organization packet	isa	the class-to-class link of a semantic net (<i>instance</i> is the individual-to-class one)
attr(s)	attribute(s)	repo	repository
freq	frequency	config	configuration
info	information	no.	number
Eg	example	Vs	compare / differentiate
w/, w/o	with, without	+Fig	draw the figure
→	leads to, therefore	=	is, equals

Reading the year tags and chips

Exam-paper month codes: Ba = Baishakh · Jth = Jestha · Shr = Shrawan · Bh = Bhadra · Ash = Ashwin · Ka = Kartik · Po = Poush · Ma = Magh · Ch = Chaitra. Years are Bikram Sambat, so 80 Ch is 2080 Chaitra.

Bold year tags are Regular exams, plain are Back exams. A header printing “Regular / Back” in one box counts as Regular.

Three course codes, three faces. Plain = CT 653 (BCT, III-II, 26 papers) · monospace = CT 710 (BEI, IV-I, 7 papers) · *italic* = CT 78506 (BEX, IV-II, 8 papers). **The same year code names different papers under different codes:** 77 Ch and 77 Ch are two different exams, and only the font tells them apart.

Tier chips over all 41 papers, 2069–2082 BS: **TOP /41** = topic asked in **8 or more** papers · **HOT /41** = **4 to 7** papers · **PIN /41** = **1 to 3** papers but worth ≥ 6 marks.

Mark chips such as **2+6** are the marks that question actually carried. **syllabus, no PYQ yet** marks a syllabus topic never yet examined but written up anyway. **verified/added** marks anything researched and added that no source note in this folder covers.